

Electrical Cabinet Library

Technical Documentation & User Manual v2026.1

Contents

- 0.1 Introduction 2
- 0.2 Layout Infrastructure 2
 - 0.2.1 Cabinet (cabinet) 2
 - 0.2.2 DIN Rails (dinrail) 2
 - 0.2.3 Wire Ducts (wireduct) 2
- 0.3 Component Library 2
 - 0.3.1 Protective Devices (mcb) 2
 - 0.3.2 Power Supply (psu) 2
 - 0.3.3 Modular Boxes (generic_box) 2
- 0.4 Advanced Wiring Techniques 2
 - 0.4.1 The Projection Principle 2
 - 0.4.2 Terminal Strips (terminal_strip) 3
 - 0.4.3 Bridging (bridge) 3
- 0.5 Style & Standardization 3
 - 0.5.1 Color Coding 3
 - 0.5.2 Anchor Naming 3

0.1 Introduction

This library provides a modular framework for creating technical 2D electrical cabinet layouts using Typst and CeTZ. It bridges the gap between manual CAD drawing and automated schematic generation.

0.2 Layout Infrastructure

0.2.1 Cabinet (cabinet)

The container for the entire assembly. It generates a grid-based coordinate system using "slots".

- **Key Anchors:** slot-1 to slot-n represent horizontal mounting levels.
- **Usage:** `#cabinet("main", (0,0), rows: 5, width: 15, height: 25).`

0.2.2 DIN Rails (dinrail)

The physical mounting surface for components.

- **Tip:** Always place rails at cabinet slot anchors for perfect alignment.
- **Example:** `#dinrail("r1", "main.slot-2").`

0.2.3 Wire Ducts (wireduct)

Used for cable management. Setting orientation: "vertical" rotates the duct correctly while maintaining logical anchors.

- **Visuals:** Includes automatic "finger" slots to simulate real-world cable entries.

0.3 Component Library

0.3.1 Protective Devices (mcb)

The Miniature Circuit Breaker represents a single-pole protection unit.

- **Anchors:** p1 (Line/Top), p2 (Load/Bottom).
- **Example:** `#mcb("F1", "rail1.west", label: "B16").`

0.3.2 Power Supply (psu)

A dynamic module that calculates its own width based on terminal count.

- **Terminals:** Supports `l_pins` (AC input) and configurable L+/M output pairs.
- **Logic:** Terminal colors are automatically applied (Red for +, Blue for -).

0.3.3 Modular Boxes (generic_box)

The foundation for complex modules like PLCs or Gateways.

- **Flexibility:** Allows pin grouping on all four sides.
- **Styling:** Groups can have individual fill colors.

0.4 Advanced Wiring Techniques

0.4.1 The Projection Principle

To avoid wires crossing through components, use the Orthogonal Projection method. This ensures wires run perfectly horizontal from the terminal into the duct.

Syntax: (Point A, "-|", Point B)

- **Effect:** Takes X from Point A and Y from Point B.
- **Code:** `wire("CB.p1", ("duct-v.east", "-|", "CB.p1")).`

0.4.2 Terminal Strips (`terminal_strip`)

Instead of placing terminals manually, use this function to generate an entire numbered block.

- **Mechanism:** It iterates through labels using `range(labels.len())`.
- **Example:** `#terminal_strip("X1", "rail2.west", ("L1", "L2", "L3", "N", "PE"))`.

0.4.3 Bridging (`bridge`)

Used for internal jumpers or busbars.

- **Function:** Connects one source to multiple targets with a single common rail.
- **Example:** `#bridge("PSU.L+", ("K1.11", "K1.A1"), offset: 0.8, stroke: red)`.

0.5 Style & Standardization

0.5.1 Color Coding

To maintain industry standards, the following stroke colors should be used.

- **Black / Dark Gray:** AC Power (230/400V).
- **Red:** DC Positive (+24V).
- **Blue:** DC Negative / Ground (0V/GND).

0.5.2 Anchor Naming

Components use standardized naming conventions for ease of use.

- **Inputs:** Always p1 or semantic names like L, A1.
- **Outputs:** Always p2 or semantic names like L+, 14.

Note: This documentation was generated for the 2026 library version. Ensure your CeTZ dependency is up to date.